

MapReduce Algorithm

Purpose & Learning Objectives

- **Purpose:** Learn the core model behind distributed processing.

Learning Objectives:

- Explain map, shuffle, and reduce as formal operators
- Estimate runtime
- Use combiners and partitioning strategies safely

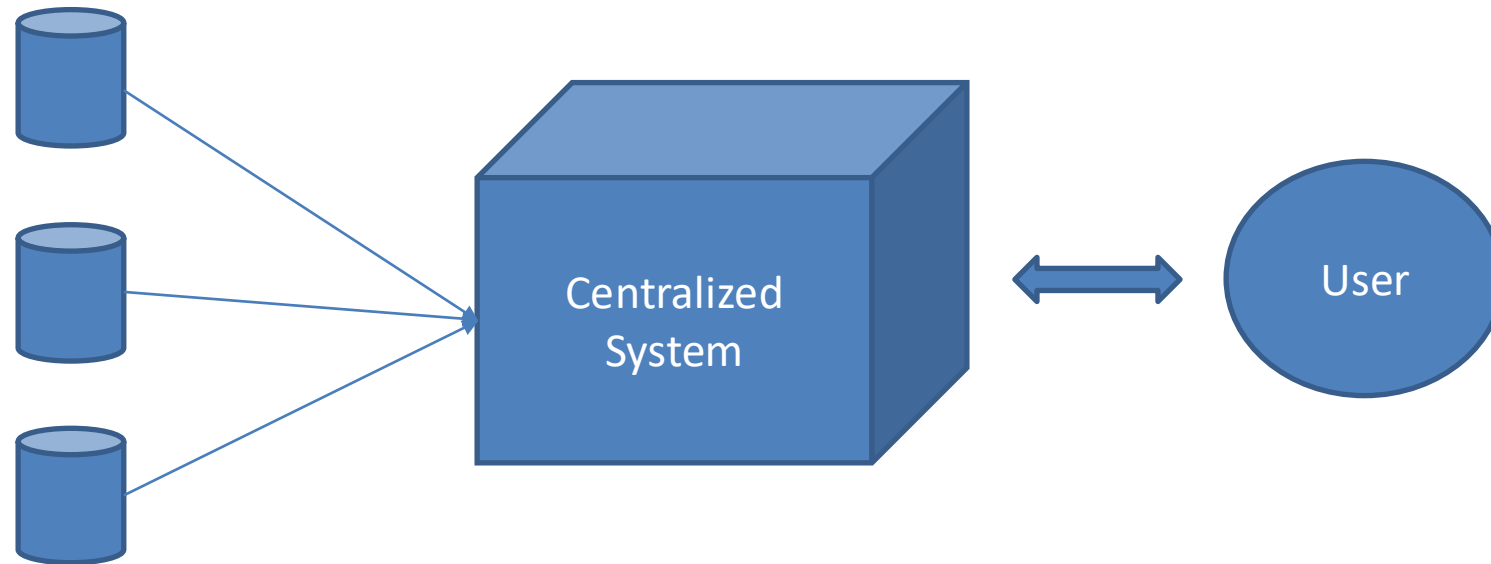
Motivation

- Process lots of data
 - Google processed about **24 petabytes** of data per day in 2009.
- **A single machine** cannot serve all the data
 - You need a distributed system to store and process **in parallel**
- Parallel programming?
 - **Threading** is hard!
 - How do you facilitate **communication** between nodes?
 - How do you **scale to more machines**?
 - How do you handle machine **failures**?

Google's Solution

Google solved this problem using an algorithm called **MapReduce**.

This algorithm divides the task into small parts and assigns them to many computers, and collects the results from them which when integrated, form the result dataset.



Google's Solution

Problem: Can't use a single computer to process the data.

Solution: Use a group of interconnected computers.

Problem: Conventional algorithms are **not** designed around memory independence.

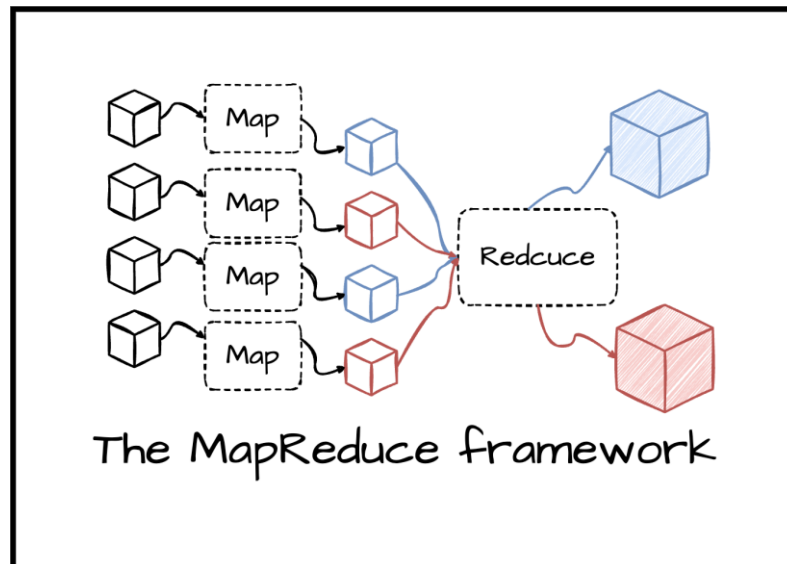
Solution: MapReduce

Definition. MapReduce is a programming paradigm model of using parallel, distributed algorithms to process or generate data sets.

How Is It Work?

The MapReduce algorithm contains two important tasks:

1. The **Map** task takes a set of data and converts it into another set of data, where individual elements are broken down into tuples.
2. The **Reduce** task takes the output from the Map as an input and combines those tuples into a smaller set of tuples.



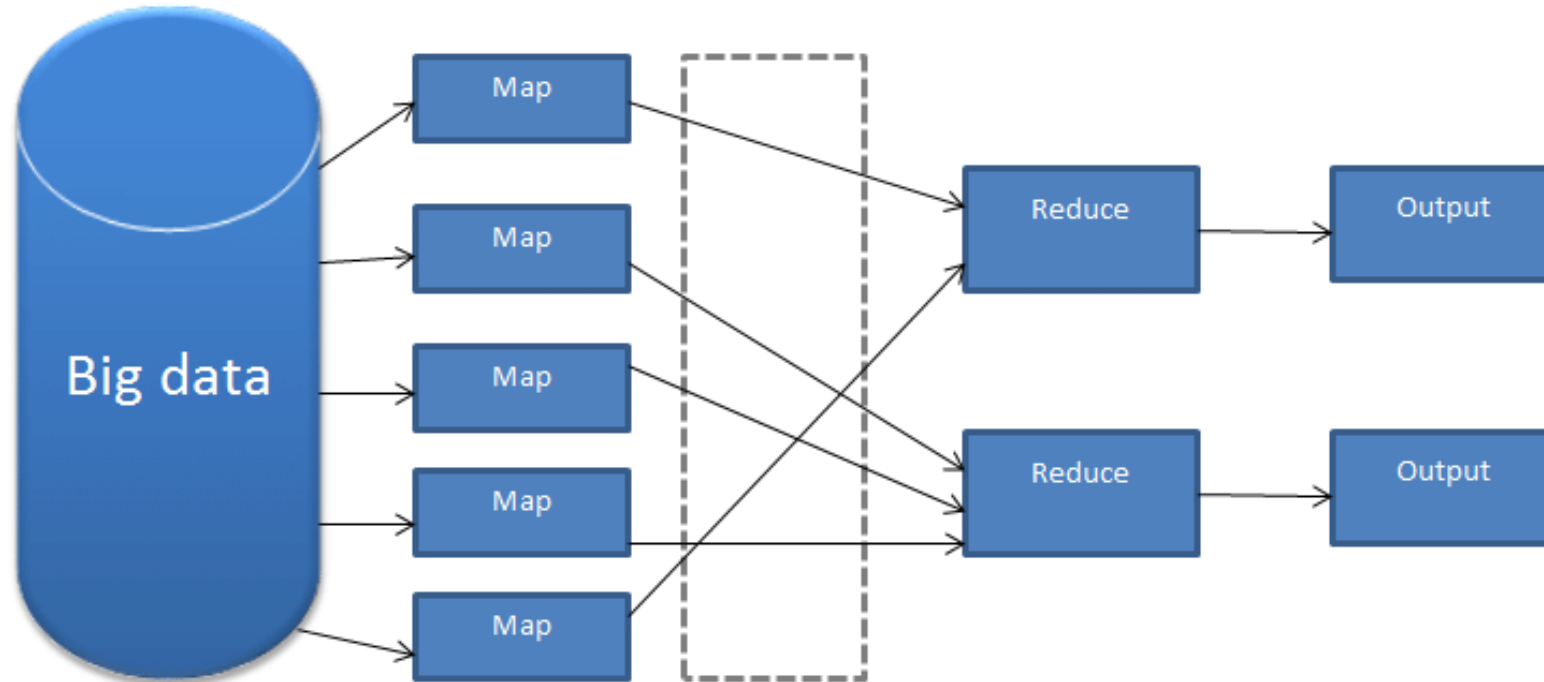
The reduce task is always performed after the map job.

Why MapReduce Matters

- Single-node processing can not handle big-data.
- MapReduce use parallel micro-batches with aggregation.
- The real “cost center” is moving and consolidating data (shuffle), not local computation.
- **SQL lens:**

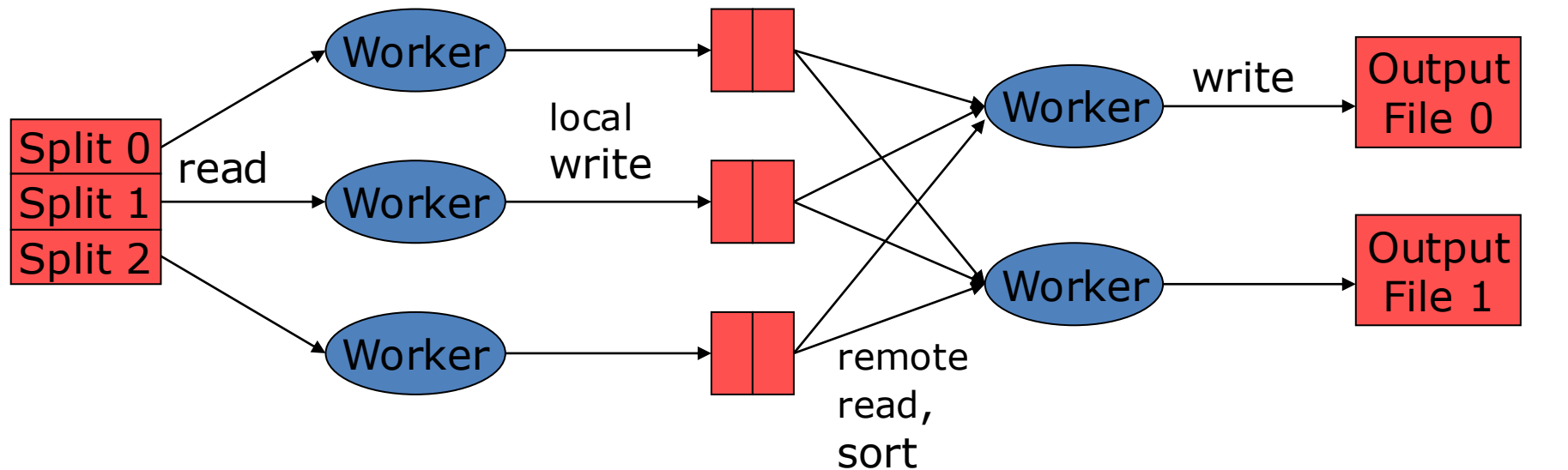
MapReduce is a physical execution model
for GROUP BY and JOIN at scale.

How Is It Work?



MapReduce workflow

Input Data



Map

extract something you
care about from each
record

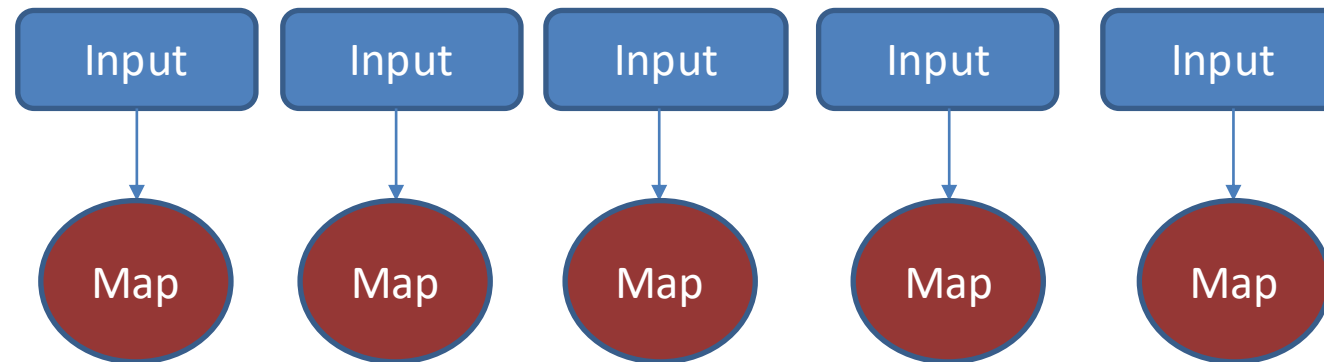
Reduce

aggregate,
summarize, filter,
or transform

MapReduce Architecture

Input Phase – Here we have a Record Reader that translates each record in an input file and sends the parsed data to the mapper.

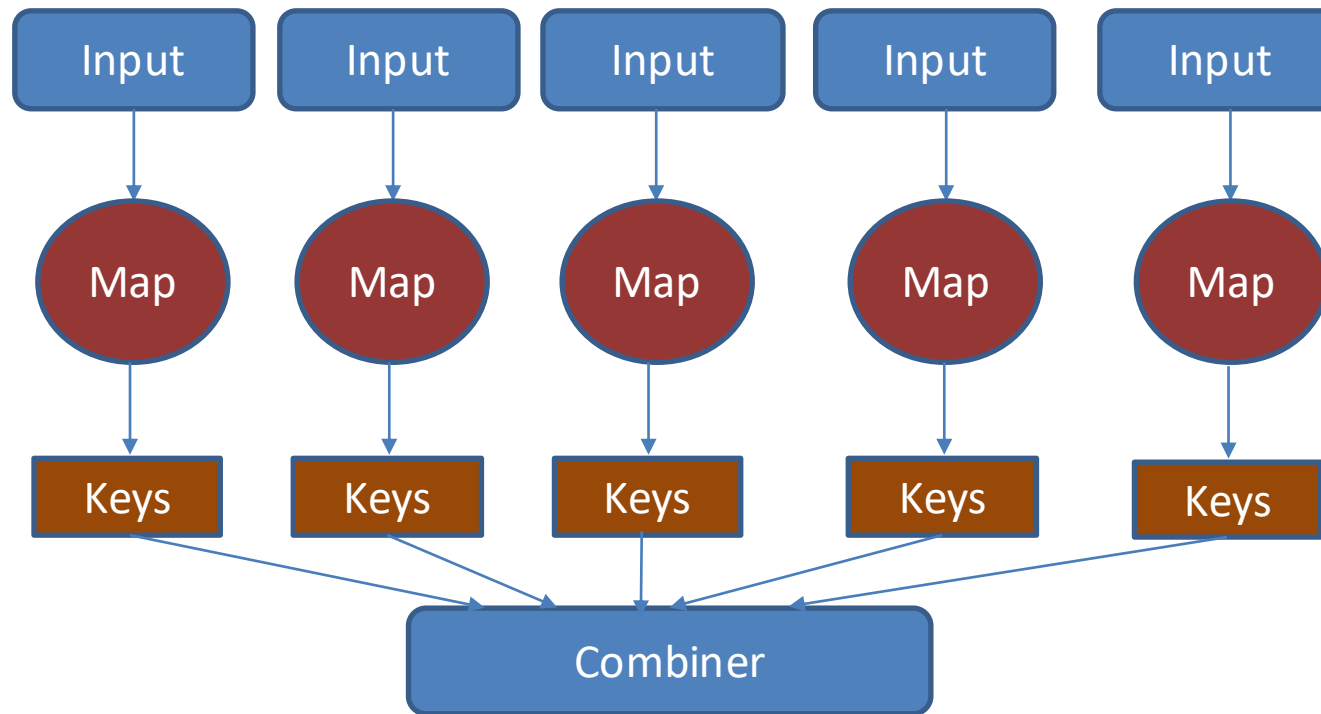
Map – Map is a user-defined function, which takes a series of key-value pairs and processes each one of them to generate zero or more key-value pairs.



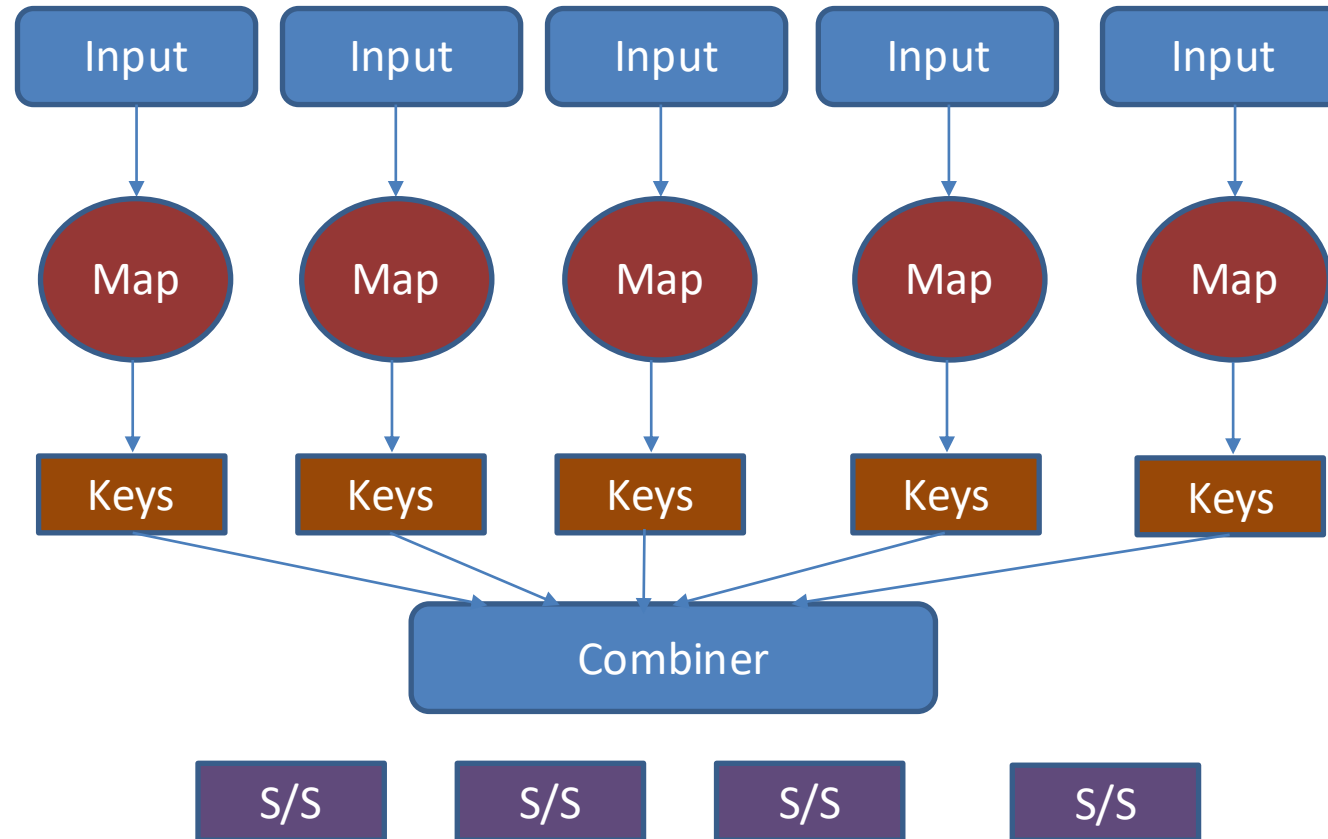
MapReduce Architecture

Intermediate Keys – They key-value pairs generated by the mapper are known as intermediate keys.

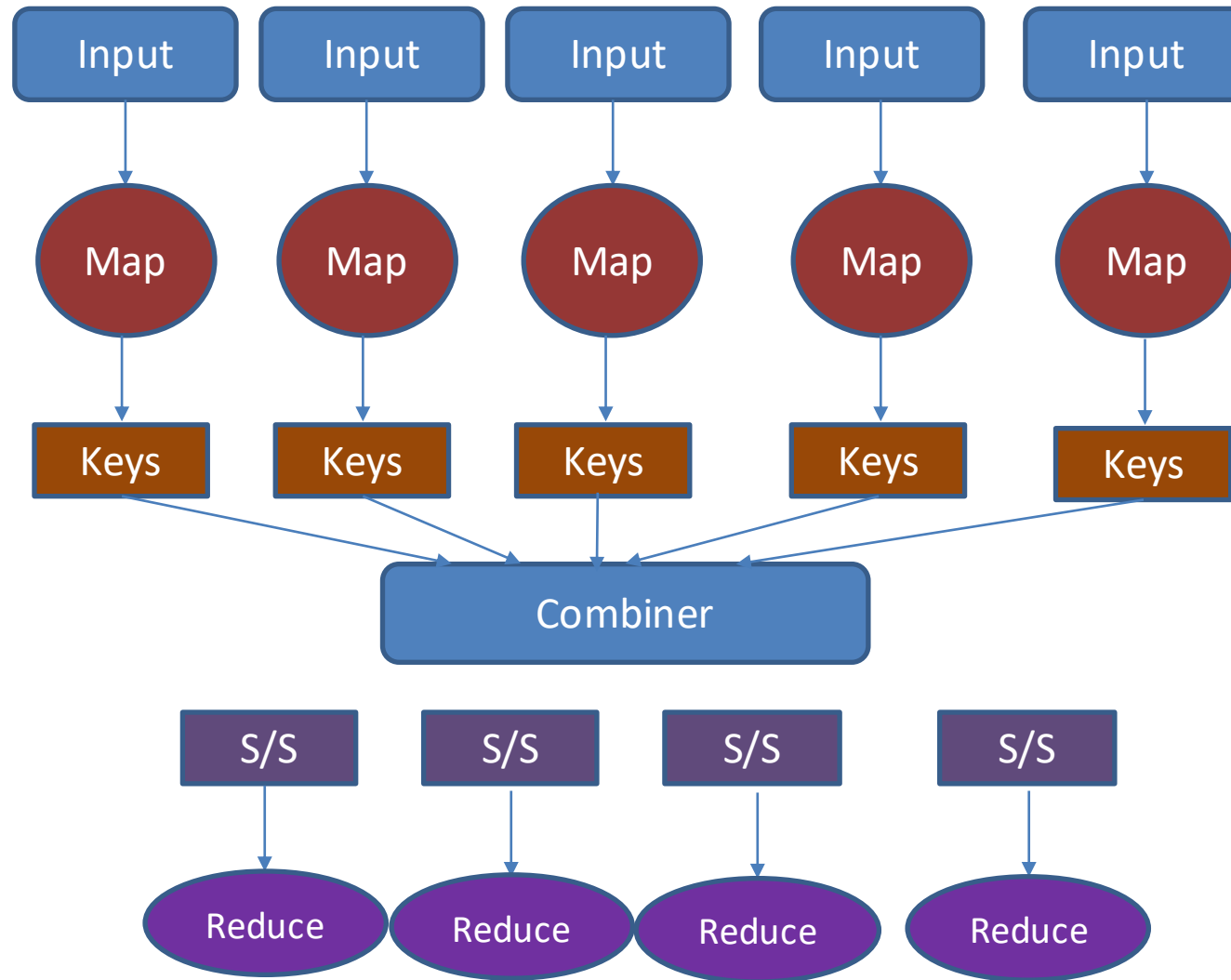
Combiner (Optional) – A combiner is a type of local Reducer that groups similar data from the map phase into identifiable sets.



MapReduce Architecture - Sorting



MapReduce Architecture - Reduce



Why Shuffle Is the Bottleneck?

- All map outputs must be **redistributed** by key.
- Network + disk spill dominate total runtime.
- CPU can be idle while waiting for shuffle.
- **SQL lens:**

This is like a global `GROUP BY` needing a full data repartition.

Example

Input

Split

Map

Shuffle & Sort

Reduce

A B R
C C R
A C B

Example

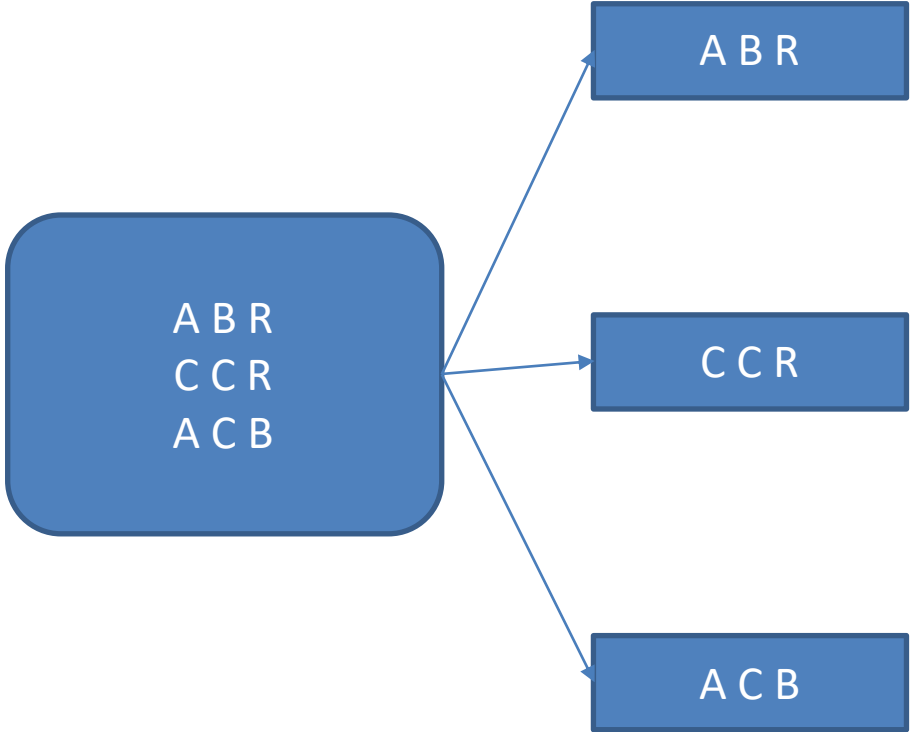
Input

Split

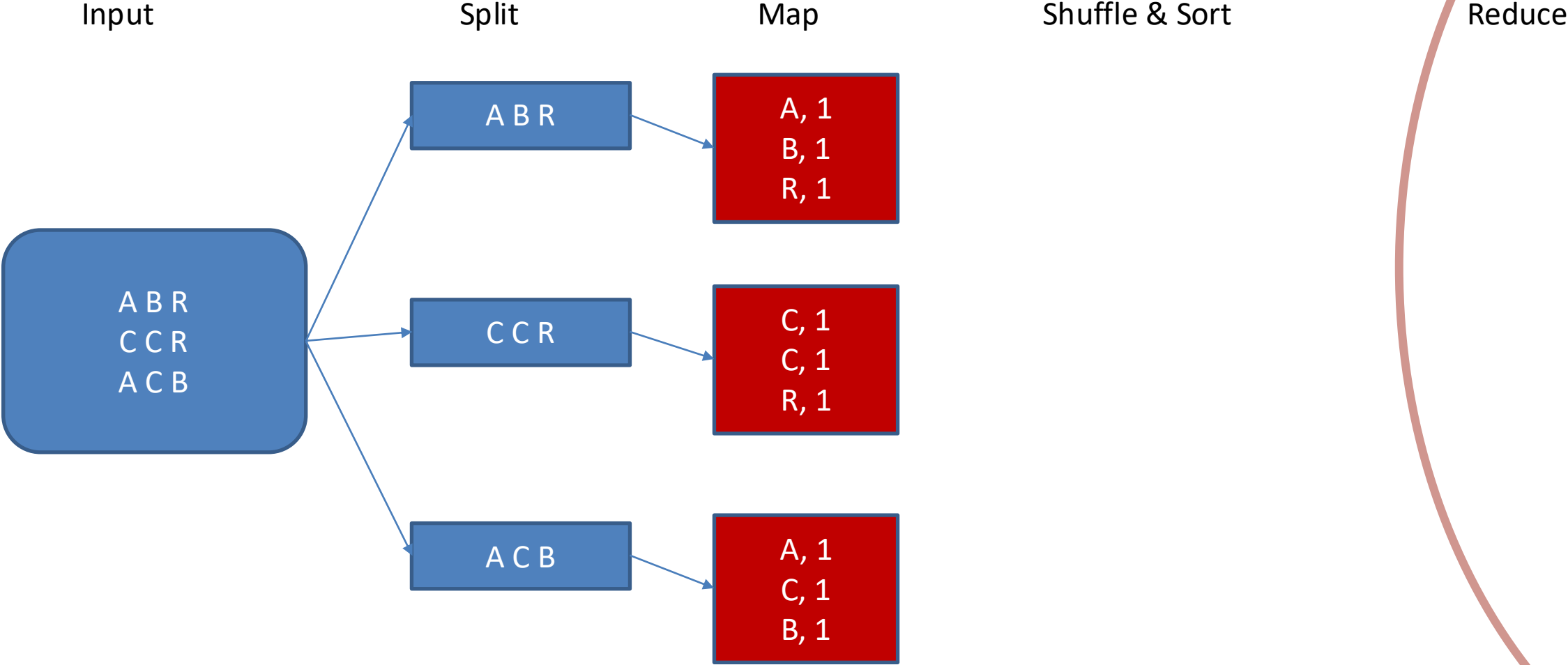
Map

Shuffle & Sort

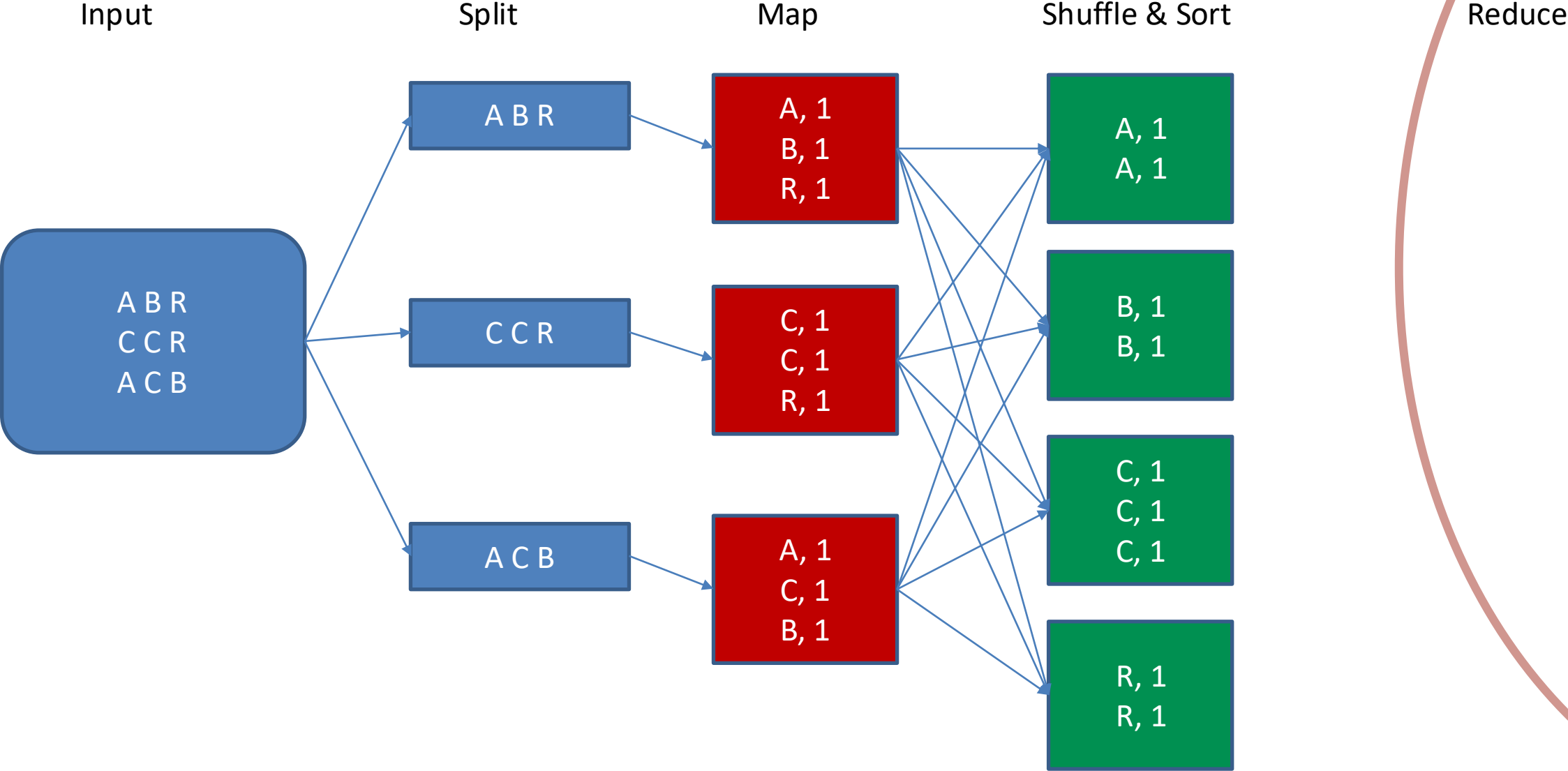
Reduce



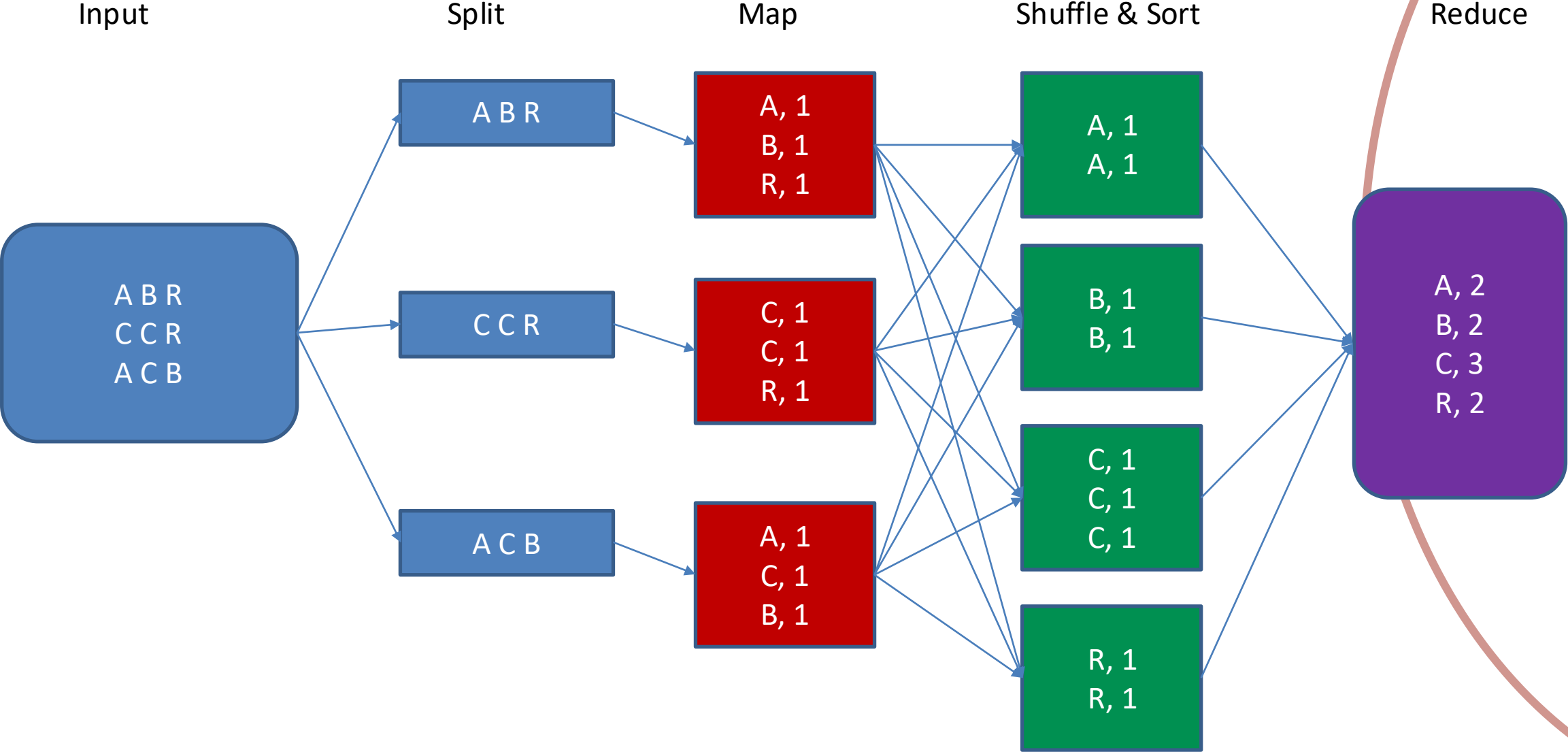
Example



Example



Example



MR Algorithm – Formal Definition

Given dataset $D \in M(K_1 \times V_1)$, where:

Symbol	Meaning
D	The input dataset.
M	A multiset. Duplicates are allowed.
K_1	The input-key domain. Example: line number, byte offset, record ID.
V_1	The input-value domain. Example: line text, order row, log event.

The mapper m transform input record $(k_1, v_1) \in K_1 \times V_1$ to

$$m : K_1 \times V_1 \rightarrow M(K_2 \times V_2)$$

The reducer r transforms grouped records into $r: K_2 \times M(V_2) \rightarrow M(K_3 \times V_3)$

Job Definition

Map all records

$$I = \bigcup_{(k_1, v_1)} m(k_1, v_1)$$

Partition

$$p(k_2) = h(k_2) \bmod R$$

Shuffle/group

$$G(k_2) = [v_2 : (k_2, v_2) \in I]$$

Reduce

$$O = \bigcup_{(k_2, v_2)} r(k_2, G(k_2))$$

Combiner rule: $f(f(A) \cup f(B)) = f(A \cup B)$

Good - sum, count, ...

Bad – median, average (naïve),...

Example

Map all records

$$I = \bigcup_{(k_1, v_1)} m(k_1, v_1)$$

Input: ABCAACC, **Mapper**: char -> (char, 1)

$$I = \{(A, 1), (B, 1), (C, 1), (A, 1), (A, 1), (C, 1), (C, 1)\}$$



Partition

$$p(k_2) = h(k_2) \bmod R$$

$$h(k_2) = \begin{cases} h(A) \% 3 = 0 \\ h(B) \% 3 = 1 \\ h(C) \% 3 = 2 \end{cases} \longrightarrow \begin{array}{l} [(A, 1), (A, 1), (A, 1)] \\ [(B, 1)] \\ [(C, 1), (C, 1), (C, 1)] \end{array}$$

Mapper

Shuffle/group

$$G(k_2) = [v_2 : (k_2, v_2) \in I]$$

$$G(A) = [1, 1, 1]$$

$$G(B) = [1]$$

$$G(C) = [1, 1, 1]$$

Reduce

$$O = \bigcup_{(k_2, v_2)} r(k_2, G(k_2))$$

$$r: (k_2, G(k_2)) \rightarrow \left(k_2, \sum_{v_2 \in G(k_2)} v_2 \right)$$

$$r(A, [1, 1, 1]) = (A, 3)$$

$$r(B, [1]) = (B, 1)$$

$$r(C, [1, 1, 1]) = (C, 3)$$

$$\longrightarrow O = \{(A, 3), (B, 1), (C, 3)\}$$

Exercises

- Given $X = [x_1, x_2, \dots, x_n]$ and $P = [P(x_1), P(x_2), \dots, P(x_n)]$ design MR algorithm to find:
 - $E[X]$
 - $V[X]$
- Given SQL table, design MR algorithm to perform WHERE of given statement.